Dated: 13/01/26

Murtaza Ali Ansari
25k-0461
BSE-3A

OOP Assignment 2

Scenario 1:

```
#include <iostream>
#include <string>
#include <vector>
#include <ctime>
using namespace std;

class registration {
private:
    string regNumber;
    string status;

public:
Registration (){
    regNumber = "";
    status = "Pending";
        }

void generateRegNumber (string areaCode){
    string letters = areaCode;
    if (letters.length() != 3) {
        letters = "KHI";
    }
```

Dated:

```
    int randomDigits = 1000 + rand() % 9000;
    regNumber = letters + to_string(randomDigits);
    status = "Registered";
}

void setStatus(string newStatus) {
    status = newStatus; }
string getRegNumber() const {
    return regNumber; }
string getStatus() const {
    return status; }
void display() const {
    cout << "Registration number: " << regNumber << " | Status:
         " << status << endl;
}
};

class Vehicle {
private:
    string make;
    string model;
    double mileage;
    Registration regRecord;

public:
Vehicle(string mk, string mdl, double mil, string areaCode)
    make = mk;
    model = mdl;
    mileage = mil;
    regRecord.generateRegNumber(areaCode);
```

Dated:

```
    }
void updateMileage (double extraMiles) {
        mileage += extraMiles;
        cout << "Mileage updated. New Mileage: " <<
                mileage << " miles." << endl;
    }

double calculateResaleValue() const {
    double baseValue = 20000.0;
    double estimateValue = baseValue - (mileage * 0.05);
    return (estimatedValue > 10000.0) ? estimatedValue : 10000.0;
}

string classifyVehicle() const {
    string num = regRecord.getRegNumber();
    if (num.length() >= 4) {
        char firstDigit = num[3];
        if (firstDigit >= '0' && firstDigit <= '4') {
            return "old";
        } else if (firstDigit >= '5' && firstDigit <= '9') {
            return "New";
        }
    }
    return "Unknown";
}
void changeRegistrationStatus(string newStatus) {
        regRecord.setStatus(newStatus);
}
```

Dated:

```
void displayVehicleDetails() const {
    cout << "- Make: " << make << ", Model: " << model << " Mileage: " << mileage << " miles" << endl;
    cout << "- Estimate Resale Value: $" << calculateResaleValue() << endl;
    regRecord.display();
    cout << "- classification: " << classifyVehicle() << endl;
}
};

class Owner {
private:
    string Name;
    string CNIC;
    vector<Vehicle> Vehicles;

public:
    Owner(string ownerName, string ownerCnic) {
        name = ownerName;
        cnic = ownerCnic;
    }

void addVehicle(const Vehicle& v) {
    vehicles.push_back(v);
    cout << "Vehicle successfully added to owner " << name << "." << endl;
}

int getVehicleCount() const {
    return vehicles.size();
}
```

Dated:

```
void displayOwnerDetails() const {
    [redacted]
    cout << "Owner Name: " << name << " | CNIC: "
         << cnic << endl;
    cout << "Total Vehicles Owned: " << getVehicleCount() << endl;
    cout << "Vehicle Details:" << endl;
    for (size_t i = 0; i < vehicles.size(); ++i) {
        cout << " [" << i + 1 << "] " << endl;
        vehicles[i].displayVehicleDetails();
    }
}
};
int main() {
    srand(time(0));
Owner owner1("Ali Khan", "42101-1234567-1");
Owner owner2("Ayesha Ahmed", "42201-1234567-1");

Vehicle v1("Toyota", "Corolla", 4500000, "KHI");
Vehicle v2("Honda", "Civic", 12000000, "LHR");
Vehicle v3("Suzuki", "Alto", 850000, "ISL");

owner1.addVehicle(v1);
owner1.addVehicle(v2);
owner2.addVehicle(v3);
cout << "\n === Displaying Details ===" << endl;
owner1.displayOwnerDetails();
owner2.displayOwnerDetails();

cout << "\n === Displaying operations ===" << endl;
```

Dated: _____

```
cout << "Updating mileage for Ali's first vehicle" << endl;
v1.updateMileage(15000);

cout << "Changing Reg Status" << endl;
v3.changeRegistrationStatus("Expired");

Scenario 2:  return 0;
}
```